SYSTEM DESIGN DOCUMENT

TaskAnalyzer | Arquitetura e contratos da Etapa Final

Projeto: sdd-task-analyzer

Objetivo: documentar os componentes, os fluxos de validacao e a execucao automatizada do TaskAnalyzer, mantendo a arquitetura simples e rastreavel ao contrato SDD.

1. Componentes da solucao

Componente	Responsabilidade	Evidencia no repositorio
src/task_analyzer.py	Validar tarefas, normalizar datas para UTC e calcular metricas gerais e por prioridade.	Funcao analyze_tasks e TaskValidationError.
tests/test_harness.py	Converter cenarios de aceite em testes repetiveis.	Oito testes de sucesso, erros e casos de borda.
GitHub Actions	Executar pytest em Python 3.11 e construir a imagem depois dos testes.	.github/workflows/ci.yml
Docker / Compose	Isolar Python e dependencias e padronizar a execucao do Test Harness.	Dockerfile e docker-compose.yml
Governanca SDD	Delimitar a IA e registrar decisoes, riscos e homologacao humana.	CONTEXT_RULES.md, specs/ e docs/.

2. Fluxo de execucao

Entrada (lista de tarefas) → validacao de tipos, IDs, status, prioridades e datas → normalizacao de timestamps para UTC → calculo de contadores, tempos e atrasos → retorno de metricas gerais e por prioridade. Dados invalidos interrompem a analise com TaskValidationError. Nao ha persistencia.

3. Contrato de metricas

Metrica	Definicao
Tempo de conclusao	Diferenca entre data_conclusao e data_inicio, em minutos. A media e calculada apenas para tarefas concluidas e tambem expressa em horas.
Taxa de atraso	Percentual de tarefas concluidas cuja data_conclusao e posterior ao prazo. Denominador: total de tarefas concluidas.
Casos sem conclusoes	Media de tempo e taxa de atraso iguais a 0.0 para evitar divisao por zero.
Agrupamento	Metricas totais e por prioridade: baixa, media e alta.

4. CI/CD e containerizacao

O workflow e executado em push para main, develop e feature/**, em pull requests destinados a main ou develop e manualmente pelo GitHub Actions. O job de testes configura Python 3.11, instala requirements.txt e executa pytest. O job de construcao da imagem depende do job de testes.

O Dockerfile utiliza python:3.11-slim, instala somente as dependencias de teste necessarias, copia o codigo e a documentacao, troca para um usuario sem privilegios e executa o Test Harness como comando padrao. O Compose monta um servico efemero de testes com filesystem somente leitura e /tmp temporario.

Comando padronizado: docker compose run --build --rm testes

5. Limites e decisoes arquiteturais

- Dependencias externas de runtime: nenhuma; pytest e usado para testes.
- Persistencia: proibida; a funcao opera somente sobre a entrada recebida.
- Timezone: timestamps precisam possuir fuso e sao convertidos para UTC.
- Validacao: IDs devem ser unicos e positivos; prioridade/status precisam pertencer aos conjuntos definidos.
- Datas cronologicamente inconsistentes geram TaskValidationError. Essa decisao esta documentada por causa do refinamento pedido na etapa intermediaria.

6. Seguranca e governanca

- O workflow solicita apenas permissao de leitura de conteudo do repositorio.
- O container executa como usuario nao-root e nao precisa montar volumes com dados do host.
- A IA nao pode ampliar o contrato, persistir dados ou adicionar bibliotecas sem autorizacao.
- A revisao humana deve comparar diff, contrato e testes antes do merge.

Documento de arquitetura gerado para acompanhar a implementacao. A execucao remota do workflow e a execucao Docker no computador de entrega devem ser confirmadas apos publicar os arquivos.